

Harvey-Grade Security and Guardrail Architecture

Detailed Technical Implementation Plan

Engineering objective

Preserve Justitia's visible specialist council while adding a machine-enforced control plane, evidence plane and evaluation plane. No legal agent should move directly from an untrusted request to an externally consequential action.

REFERENCE DATE

14 September 2026

TARGET STACK

React · TypeScript · tRPC · Drizzle · MySQL/TiDB

Contents

1. Executive implementation decision	1
2. Verified current-state anchors	1
3. Target architecture	3
3.1 Control Plane	4
3.2 Execution Plane	4
3.3 Evidence Plane	4
3.4 Assurance Plane	4
4. Deployment and execution options	4
5. Phase 0 — Freeze and threat-model the system	5
6. Phase 1 — Identity, tenancy and matter isolation	6
6.1 Protect all legal procedures	6
6.2 Add the organisation and matter security model	6
6.3 Enforce ownership in the data layer	6
6.4 Close the Higgins bridge	7
6.5 Phase 1 acceptance criteria	7
7. Phase 2 — Data classification and provider policy	7
7.1 Classification model	7
7.2 Provider capability registry	8
8. Phase 3 — Versioned Agent Manifests	8
8.1 Publication workflow	9
9. Phase 4 — Plan-before-execution and approval gates	9
9.1 Split the current <code>submitTask</code>	9
9.2 Execution-plan contract	9
9.3 Risk and approval matrix	10
9.4 Exact execution sequence	10
10. Phase 5 — Policy Decision Point and default-deny Tool Gateway	10
10.1 Policy architecture	10
10.2 Tool Gateway	11
10.3 Capability tokens	11
11. Phase 6 — Isolated execution and resource budgets	11
11.1 Minimum isolation for the current application	12
11.2 Strong isolation for a dedicated worker	12
12. Phase 7 — Evidence Ledger and citation verification	12
12.1 Evidence-first data model	12
12.2 Validation pipeline	12
13. Phase 8 — Tamper-evident audit and privacy-aware observability	13

14. Phase 9 — Governed memory and Karpathy Loop	13
15. Phase 10 — JLC Eval Lab and Guardrails simulation	14
15.1 Benchmark families	14
15.2 Evaluation method	14
15.3 Synthetic-user simulations	15
16. Phase 11 — Release gates, canaries and rollback	15
17. Scheduled assurance and operational jobs	15
18. Concrete module and file plan	16
18.1 New server modules	16
18.2 Existing modules to refactor	16
18.3 Frontend work	17
19. Migration sequence	17
20. Test strategy	17
20.1 Unit tests	17
20.2 Integration tests	18
20.3 Adversarial tests	18
20.4 Recovery tests	18
20.5 Minimum continuous-integration gate	18
21. Proposed security and quality dashboard	18
22. Responsibility model	19
23. Ninety-day delivery plan	19
24. Definition of done	19
25. Final engineering guidance	20
References	20

1. Executive implementation decision

Harvey's transferable advantage is not one model. It is the governed system around the models: plan inspection, matter boundaries, least privilege, execution traces, cited evidence, adversarial testing and formal assurance.^{1 2 3 4} Its acquisition of Guardrails AI reinforces the importance of simulation and systematic failure discovery for long-horizon agents.⁵

Justitia already contains valuable ingredients: named specialists, explicit task routing, primary/fallback models, legal methods, Trust Gate prompt instructions, entity memory, the Practice Profile, Board escalation and a Karpathy-style learning loop. The implementation gap is that many controls remain prompt-based, post-hoc or duplicated across ingress paths.

The target architecture therefore introduces six mandatory separations:

Separation	Required outcome
Identity from business logic	Every call has an authenticated human or service principal, organisation, matter and role before legal logic runs.
Planning from execution	A model may propose a plan without receiving authority to execute it.
Policy from prompts	Non-bypassable code decides what may be read, called, written, sent or learned.
Tools from agents	Agents receive short-lived capabilities through a default-deny gateway rather than direct credentials.
Evidence from prose	Material claims must reference immutable evidence records and pass independent checks.
Learning from production	No extracted lesson becomes active memory until provenance, review, evaluation, versioning and rollback are complete.

The implementation should begin with containment and authorisation. Additional autonomy, enterprise integrations and proprietary model training come later.

2. Verified current-state anchors

The following plan is mapped to the inspected Justitia code rather than to a hypothetical greenfield system.

Current component	Verified behaviour	Required change
server/routers/legal.ts	submitTask is a publicProcedure; it creates a task, invokes a model, then runs detectHITL after generation. A client can also provide modelOverride.	Replace direct execution with authenticated plan creation, policy decisions, approval and queued execution. Remove unrestricted model override.
server/routers/higginsBridge.ts	The bridge permits all requests when HIGGINS_BRIDGE_TOKEN is absent. It duplicates the model-execution and post-hoc HITL path.	Fail closed, require a scoped service identity, and route through the same control-plane service as tRPC.
server/legal/agents.ts	Agent roles, models, fallbacks, prompts, Trust Gate and heuristic HITL rules are defined in code.	Convert these ingredients into immutable, versioned Agent Manifests with machine-enforced permissions.
server/routers/dossier.ts	Dossier CRUD and audit access are predominantly public; audit rows are ordinary mutable database records.	Enforce matter ACLs and introduce append-only, hash-chained security events.

Current component	Verified behaviour	Required change
server/legal/karpathyEngine.ts	Lessons can be extracted fire-and-forget from completed outputs and inserted into entity memory.	Quarantine proposed lessons until human approval and regression checks succeed.
drizzle/schema.ts	The inspected legal tables do not establish organisation, tenant, owner or matter-level access-control relationships.	Add organisation, membership, matter, ACL and ownership keys to all legal resources.

This plan is a technical design and targeted implementation roadmap. It is not a penetration test, certification opinion or legal conclusion about privilege, professional conduct or regulatory compliance.

3. Target architecture

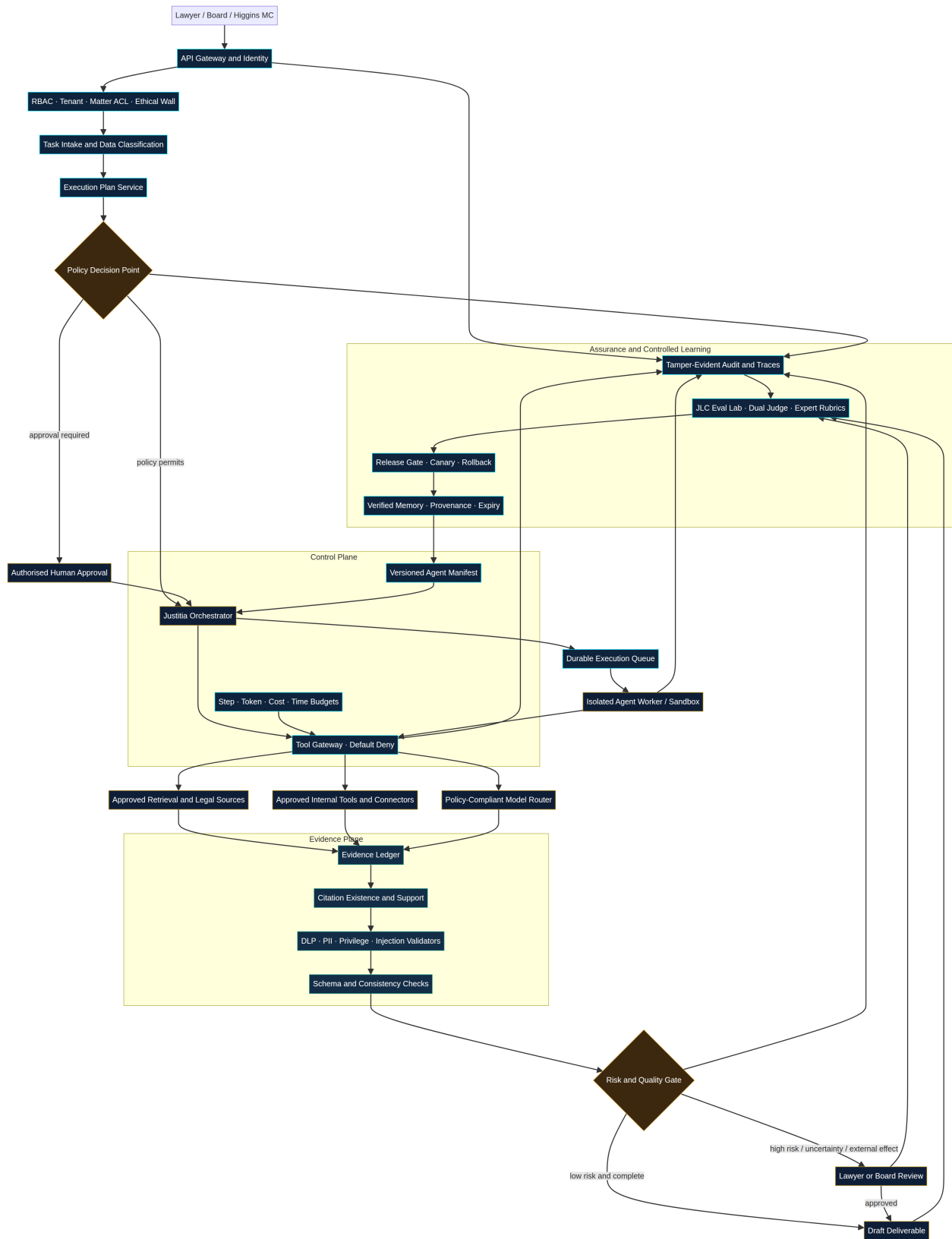


Figure 1: Justitia Harvey-grade guardrail architecture

The architecture has one controlling rule: **the orchestrator never owns authority**. It requests authority from the Policy Decision Point. The same rule applies to every specialist, fallback model and external integration.

3.1 Control Plane

The Control Plane owns identity, matter boundaries, data classification, plan generation, policy decisions, approvals, agent manifests, budgets and tool capabilities. It must be deterministic wherever a rule can be deterministic.

3.2 Execution Plane

The Execution Plane consumes only approved, immutable plans. It uses an isolated worker, has no unrestricted credentials, and receives time-limited capability tokens for individual tools. A plan approval is not blanket authority: every material tool invocation is re-authorised against the current matter policy.

3.3 Evidence Plane

The Evidence Plane stores provenance before prose. Retrieval results, source passages, content hashes, jurisdiction, dates and access labels become immutable evidence records. The final output refers to those records through claim-level citations.

3.4 Assurance Plane

The Assurance Plane stores traces, runs deterministic validators, evaluates legal work products, blocks unsafe releases and governs memory changes. It also supports forensic review without logging full privileged content unnecessarily.

4. Deployment and execution options

Two viable deployment patterns should remain available until expected concurrency, threat model and document workload are measured. The lighter pattern accelerates containment; the isolated pattern provides the stronger long-term security boundary.

Approach	Trade-offs	Cost	Setup complexity
A. Harden the current managed application	Keep planning, policy checks and bounded agent execution request-scoped in the existing TypeScript service. Use durable database state and scheduled HTTP callbacks for recurring evaluations. Fastest delivery and lowest operational overhead, but process-level isolation is weaker and long document jobs must remain within platform limits.	Low	Low-medium
B. Split Control Plane and isolated execution service	Keep identity, matters, policy and approval in the current application; dispatch approved plans to a dedicated worker that supports container or micro-VM isolation, network egress controls and independent scaling. Stronger confidentiality and	Medium-high	High

Approach	Trade-offs	Cost	Setup complexity
	tool isolation, but requires queue operations, worker hardening and more complex incident response.		

Both approaches start with the same schema, policy model, Agent Manifest and evidence contract. This prevents architectural rework if Justitia moves from the lighter pattern to isolated workers. The decision trigger is not feature count; it is the combination of confidential data class, untrusted file handling, external tool effects, concurrency and execution duration.

In-process timers such as `setInterval` and `node-cron` must not be used for security regression, retention or evaluation jobs. Use the platform's scheduled HTTP callbacks with authenticated cron identity, idempotent handlers and durable job ownership.

5. Phase 0 — Freeze and threat-model the system

Duration: 2–3 working days.

Release condition: no production autonomy expansion during this phase.

The first deliverable is a threat model covering actors, assets, trust boundaries and failure modes. It must include external users, internal users, administrators, Higgins MC, model providers, legal-content providers, uploaded files, compromised accounts, malicious documents and a compromised specialist agent.

Threat class	Required control owner	Minimum test
Cross-tenant or wrong-matter access	Identity and Policy Decision Point	A user from Organisation A cannot enumerate, retrieve, cite or infer Organisation B data.
Prompt injection in documents or web sources	Intake, retrieval and validators	Embedded instructions cannot change the approved plan, tools, system rules or destination.
Tool misuse	Tool Gateway	An agent cannot call an unapproved tool, destination, method or argument pattern.
Privilege leakage	Evidence and output validators	Privileged content cannot enter logs, telemetry, memory or an unauthorised response.
Unsupported legal conclusion	Citation and quality validators	Every material claim links to a source passage that exists and supports the claim.
Approval bypass	Policy and approval service	Direct API, Higgins bridge, retries and fallback models cannot execute a gated action without valid approval.
Memory poisoning	Controlled-learning service	Unreviewed lessons remain quarantined and cannot affect prompts.
Provider data-policy mismatch	Model router	Confidential or privileged jobs cannot use a provider/region that violates the configured retention policy.

Create a release freeze checklist in the repository. Every later phase must show which threat cases it closes and which remain open.

6. Phase 1 — Identity, tenancy and matter isolation

Duration: 5–8 working days.

Priority: Critical.

6.1 Protect all legal procedures

Replace `publicProcedure` with `protectedProcedure` for every legal task, dossier, memory, configuration, audit and agent-chat operation that touches non-public information. Administrative configuration and agent publication must use `adminProcedure` or a new permission-aware equivalent.

Do not rely on the frontend to hide unauthorised actions. All access decisions must occur in the server procedure and again in the data-access helper.

6.2 Add the organisation and matter security model

Introduce the following core tables using Drizzle migrations and apply the generated SQL through the managed migration path.

Table	Essential fields	Security purpose
<code>organisations</code>	<code>id, name, status, dataRegion, createdAt</code>	Root tenant boundary.
<code>organisation_members</code>	<code>organisationId, userId, role, status</code>	Membership and organisation role.
<code>matters</code>	<code>id, organisationId, name, riskTier, jurisdiction, status, createdBy</code>	Confidential legal-work boundary.
<code>matter_members</code>	<code>matterId, userId, role, canApprove, canExport</code>	Matter-level RBAC.
<code>matter_acl</code>	<code>matterId, principalType, principalId, resourceType, actions, conditions</code>	Fine-grained resource authorisation and ethical walls.
<code>service_principals</code>	<code>id, organisationId, name, status, credentialVersion</code>	Scoped machine identity for Higgins and future integrations.

Every legal resource must carry `organisationId` and `matterId`. This includes tasks, dossiers, documents, execution plans, evidence, output versions, memory items, audit events and evaluations. A nullable matter key should be limited to explicitly global configuration that is administered separately.

6.3 Enforce ownership in the data layer

Create query helpers that require an `AccessScope` object rather than accepting only a row identifier.

```
type AccessScope = {
  userId: number;
  organisationId: number;
  matterId: number;
  permissions: Set<Permission>;
};

async function getLegalTask(scope: AccessScope, taskId: number) {
  // WHERE id = taskId AND organisation_id = scope.organisationId
  // AND matter_id = scope.matterId
}
```

No router may call a generic `getById` helper for a legal resource. Add unit tests that fail when tenant or matter filters are absent.

6.4 Close the Higgins bridge

Delete the open-access branch that permits requests when HIGGINS_BRIDGE_TOKEN is absent. Startup must fail in production when no integration credential is configured.

The replacement should use a scoped service principal with the following controls:

Control	Requirement
Authentication	Signed short-lived token or mutually authenticated service connection.
Scope	Explicit organisation, permitted matters, endpoints and task types.
Replay defence	Timestamp, nonce and bounded clock skew.
Rotation	Versioned credentials with overlap during rotation and immediate revocation.
Rate limits	Per principal, organisation and endpoint.
Audit	Log principal ID, policy result, request hash and correlation ID; do not log full privileged payload.

The bridge must call the same application service as tRPC. It may not maintain a second orchestration implementation.

6.5 Phase 1 acceptance criteria

Criterion	Required result
Unauthenticated legal request	401 UNAUTHORIZED
Authenticated user outside organisation	403 FORBIDDEN without existence disclosure
Organisation member outside matter	403 FORBIDDEN
Cross-tenant ID enumeration	Zero accessible records across automated tests
Missing Higgins credential in production	Startup or route registration fails closed
Legal configuration mutation by ordinary user	403 FORBIDDEN

7. Phase 2 — Data classification and provider policy

Duration: 4–6 working days.

7.1 Classification model

Every task, document and evidence item receives one data class at intake.

Data class	Typical example	Minimum provider rule
PUBLIC	Published judgment or statute	Approved provider; ordinary retention policy permitted.
INTERNAL	Internal template without sensitive client facts	Approved provider and organisation policy.
CONFIDENTIAL	Commercial contract or investigation material	Zero-training commitment; configured retention and region.
PRIVILEGED	Legal advice, litigation strategy or counsel communication	Strongest approved provider profile; no secondary use; strict logging and export controls.
REGULATED	Special-category personal or medical data	Jurisdiction, residency and purpose restrictions; explicit policy decision.

Classification should combine deterministic rules, user confirmation and a bounded classifier. The model classifier proposes a label; code applies policy. A lower-trust input can never downgrade an existing classification.

7.2 Provider capability registry

Replace free-form `modelOverride` with an administrator-maintained provider registry.

Registry field	Example purpose
<code>provider, model, version</code>	Immutable routing identity.
<code>allowedDataClasses</code>	Prevent privileged jobs using an unsuitable endpoint.
<code>regions</code>	Enforce organisation residency policy.
<code>retentionProfile</code>	Express zero-retention or approved retention conditions.
<code>capabilities</code>	Context length, structured output, tool use, vision and citation support.
<code>evaluationStatus</code>	Block unapproved or regressed models.
<code>effectiveFrom, retiredAt</code>	Support controlled lifecycle and rollback.

Model selection becomes a policy decision based on task type, jurisdiction, data class, document size, cost budget and benchmark performance. Users may request an allowed routing preference, but cannot bypass policy.

8. Phase 3 — Versioned Agent Manifests

Duration: 5–7 working days.

Convert each `AgentConfig` into an immutable, versioned `AgentManifest`. A manifest is code-enforced policy, not merely prompt text.

```
const AgentManifestSchema = z.object({
  id: z.string(),
  version: z.string(),
  status: z.enum(["draft", "testing", "approved", "retired"]),
  role: z.string(),
  taskTypes: z.array(LegalTaskTypeSchema),
  jurisdictions: z.array(z.string()),
  primaryModelPolicy: z.string(),
  fallbackModelPolicy: z.string(),
  allowedTools: z.array(z.string()),
  allowedSourceClasses: z.array(z.string()),
  allowedDataClasses: z.array(DataClassSchema),
  maxSteps: z.number().int().positive(),
  maxToolCalls: z.number().int().nonnegative(),
  maxTokens: z.number().int().positive(),
  maxCostMinor: z.number().int().nonnegative(),
  timeoutSeconds: z.number().int().positive(),
  approvalPolicyId: z.string(),
  outputSchemaId: z.string(),
  validatorSuiteId: z.string(),
  evalSuiteId: z.string(),
})
```

```
promptTemplateHash: z.string(),
});
```

Store the complete manifest, content hash, creator, approver and lifecycle status. A production run references an exact manifest version. Editing an agent creates a new draft version; it never mutates an approved version.

8.1 Publication workflow

State	Allowed action
draft	Edit and run local/unit tests only.
testing	Run sandboxed eval suites and adversarial simulations.
approved	Execute production tasks under its declared permissions.
retired	No new runs; historical replay remains possible.

Publishing requires an authorised administrator and evidence that required regression, security and legal-quality thresholds passed.

9. Phase 4 — Plan-before-execution and approval gates

Duration: 6–9 working days.

9.1 Split the current submitTask

Replace the synchronous “submit and execute” mutation with four explicit operations.

New operation	Responsibility
legal.createPlan	Validate access, classify data, select a manifest, generate a bounded plan and store risk indicators.
legal.approvePlan	Authorised reviewer approves, rejects or narrows scope; stores a signed decision and expiry.
legal.executePlan	Revalidates policy and queues an immutable approved plan.
legal.reviewOutput	Records approval, requested revision, rejection and permitted release destination.

The existing submitTask should become a compatibility wrapper for low-risk internal tasks only. It must call the same plan service and can auto-approve only when deterministic policy says no approval is required.

9.2 Execution-plan contract

An ExecutionPlan must contain:

Field group	Required content
Identity	Organisation, matter, requesting principal and agent manifest version.
Objective	User objective and machine-readable task type.
Scope	Documents, systems, jurisdictions and permitted date range.
Steps	Ordered steps with expected inputs, outputs and maximum retries.
Tools	Exact tools and permitted argument constraints.
Sources	Source classes and jurisdictional hierarchy.
Budgets	Steps, tool calls, tokens, time and cost.
Risk	Data class, action class, uncertainty and required approvals.
Integrity	Content hash, creation time, expiry and superseded plan ID.

9.3 Risk and approval matrix

Risk tier	Example	Minimum gate
R0	Public-law research with no client facts and no external effect	Policy auto-approval; post-run quality gate.
R1	Internal draft summary or clause extraction	User plan confirmation; no external send.
R2	Confidential contract advice, personal data or material commercial impact	Qualified legal reviewer approval before execution or release.
R3	Filing, regulator communication, final red-line, settlement, privileged strategy, employment termination or high-impact Board decision	Dual approval or designated Board/legal authority; external action remains separate.

Approval must be bound to the plan hash. Any material plan change, new tool, new document, new destination or classification upgrade invalidates prior approval.

9.4 Exact execution sequence

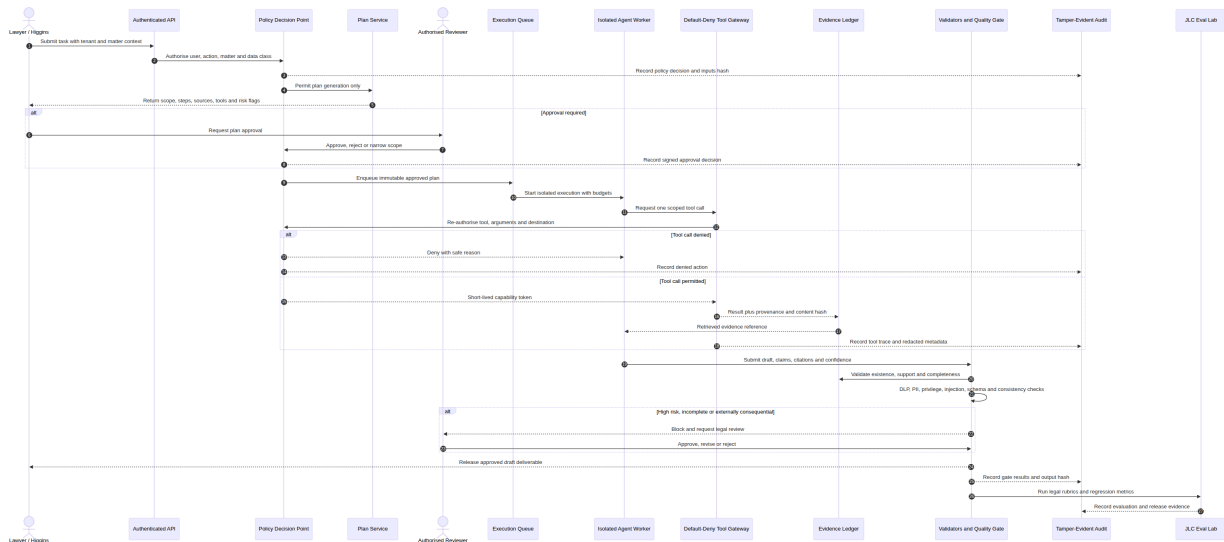


Figure 2: Justitia guarded execution sequence

The Policy Decision Point appears twice by design: first for the plan, then for each tool request. This prevents an approved general objective from silently expanding into unapproved actions.

10. Phase 5 — Policy Decision Point and default-deny Tool Gateway

Duration: 8–12 working days.

10.1 Policy architecture

Start with a typed TypeScript policy interface so enforcement can be added without introducing an external runtime. Keep the interface compatible with a later policy engine if rule volume or audit requirements justify one.

```
type PolicyRequest = {
  principal: PrincipalContext;
```

```

organisation: OrganisationPolicy;
matter: MatterPolicy;
agent: AgentManifest;
plan: ExecutionPlan;
action: "plan" | "execute" | "tool" | "release" | "learn";
resource?: ResourceDescriptor;
toolRequest?: ToolRequest;
};

type PolicyDecision = {
  effect: "allow" | "deny" | "require_approval";
  reasonCodes: string[];
  obligations: PolicyObligation[];
  expiresAt: number;
  decisionHash: string;
};

```

Policy obligations can require redaction, a specific provider region, a shorter retention period, a second approver, source restrictions or a lower budget.

10.2 Tool Gateway

Agents must never import integration clients directly. Register each tool with a deterministic contract.

Tool contract element	Requirement
Tool identity	Stable ID and version.
Input schema	Strict Zod schema with length, enum and pattern bounds.
Output schema	Strict shape; unexpected content is rejected or quarantined.
Allowed agents	Exact manifest IDs or capability class.
Allowed matters/data	Policy-controlled data classes and destinations.
Network policy	Host and method allowlist; no arbitrary URLs or redirects.
Side-effect class	read, draft, modify, external_send or file.
Approval requirement	Determined by action, matter and risk tier.
Idempotency	Required for writes and retries.
Audit policy	Redacted metadata, hashes, timing and result status.

External send, filing, record modification and deletion tools should not exist in the first production Tool Gateway version. Begin with read-only retrieval and internal draft generation.

10.3 Capability tokens

When policy permits a call, issue a single-use, short-lived capability token containing the run, matter, tool, permitted arguments hash, expiry and nonce. The Tool Gateway validates the token and consumes it atomically. This prevents a compromised worker from reusing broad application credentials.

11. Phase 6 — Isolated execution and resource budgets

Duration: 8–15 working days, depending on deployment option.

11.1 Minimum isolation for the current application

Under the lighter deployment pattern, every run still receives a unique execution ID, immutable plan snapshot, bounded temporary workspace and strict tool interface. No tool credential is placed in the model prompt or returned to the browser. All state required after the request is stored durably.

11.2 Strong isolation for a dedicated worker

For confidential multi-document work or untrusted files, use a dedicated worker with the following constraints:

Boundary	Required control
Filesystem	Ephemeral workspace mounted only for one run; read-only source bundle where possible.
Network	Deny by default; egress only through the Tool Gateway or explicit proxy allowlist.
Credentials	No static provider or storage credentials; short-lived scoped capabilities only.
Resources	CPU, memory, wall time, process, token, step, tool-call and cost budgets.
Concurrency	Per-organisation and per-matter limits to prevent denial of service.
Output	Only schema-valid evidence, trace and draft objects can leave the worker.
Destruction	Workspace is destroyed after evidence and trace persistence; destruction is audited.

The worker must assume that prompts, retrieved content and generated code are hostile. Shell, package installation and arbitrary network tools remain unavailable unless a separate high-risk agent class is explicitly designed and approved.

12. Phase 7 — Evidence Ledger and citation verification

Duration: 8–12 working days.

12.1 Evidence-first data model

Table	Essential content
evidence_items	Source type, source URI, document ID, passage, page/paragraph, jurisdiction, authority level, publication date, retrieved time, content hash and access label.
claims	Output version, exact claim text, materiality, jurisdiction, confidence and review status.
claim_evidence_links	Claim-to-evidence relationship and support type: supports, contradicts, context or not_found.
citation_checks	Existence, quote accuracy, authority, freshness, support and validator version.
validation_findings	Severity, rule ID, location, evidence and remediation status.

12.2 Validation pipeline

Every draft passes the following stages in order:

Stage	Method	Blocking condition
Structural validity	Deterministic schema validation	Missing required section or invalid output type.
Citation existence	Deterministic source lookup	Source or passage cannot be located.
Citation entailment	Independent model plus deterministic quotation checks	Material claim is unsupported or contradicted.
Source hierarchy	Jurisdictional rules	Secondary source used where a required primary authority is available.
Conflict detection	Compare evidence set	Material sources conflict without acknowledgement.
DLP and privacy	Deterministic patterns plus bounded classifier	Unauthorised PII, secrets or privileged material in destination.
Prompt-injection trace	Retrieval-content classifier and instruction boundary	Untrusted source attempted to alter system, plan, tools or destination.
Legal-risk gate	Rules plus manifest policy	High-risk conclusion lacks required reviewer approval.

Material claims must be explicit objects before prose release. A response with fluent citations but no verified claim-to-passage mapping is not considered sourced.

13. Phase 8 — Tamper-evident audit and privacy-aware observability

Duration: 5–7 working days.

Create an append-only `security_audit_events` table with:

Field	Purpose
<code>eventId, sequence</code>	Stable ordering within organisation and run.
<code>organisationId, matterId, runId</code>	Security scope.
<code>actorType, actorId</code>	Human, service, agent, system or reviewer.
<code>eventType, reasonCodes</code>	Machine-readable event and decision basis.
<code>inputHash, outputHash</code>	Integrity without storing full sensitive content.
<code>previousEventHash, eventHash</code>	Hash chain for tamper evidence.
<code>policyVersion, manifestVersion, validatorVersion</code>	Reproducibility.
<code>redactedMetadata, createdAt</code>	Operational context with data minimisation.

Audit access is itself matter-scoped and logged. Privileged prompt content, source passages and full outputs should not be copied into general application logs. Use correlation IDs and hashes to join security events to protected evidence stored under its own ACL.

Define alerts for repeated denials, cross-matter access attempts, approval bypass, capability-token replay, unusual export volume, provider-policy mismatch, validator failure and audit-chain discontinuity.

14. Phase 9 — Governed memory and Karpathy Loop

Duration: 5–8 working days.

Replace direct memory activation with a staged lifecycle.

Memory state	Behaviour
proposed	Extracted lesson is quarantined; it cannot influence any prompt.
validated	Provenance, source access and jurisdiction are confirmed.
approved	Authorised reviewer accepts scope, wording, expiry and affected agents.
active	Item can be retrieved under tenant, matter and jurisdiction policy.
superseded	New version replaces the item; historical runs remain reproducible.
revoked	Item is immediately excluded; reason and affected runs are recorded.

Each memory item requires source task, source output hash, human approver, jurisdiction, legal-effective date, expiry/review date, confidence, sensitivity, target agents and rollback pointer.

Before activation, run regression cases that are likely to retrieve the proposed memory. If the item degrades citation support, issue spotting, neutrality or jurisdictional accuracy, activation fails.

15. Phase 10 — JLC Eval Lab and Guardrails simulation

Duration: 10–20 working days for the first production suite.

Harvey’s Legal Agent Benchmark repository contains tasks, rubrics, sandboxing and evaluation components under the MIT licence.⁶ Justitia can reuse the harness architecture while preserving the licence notice and replacing the legal content with its own authorised benchmark cases.

15.1 Benchmark families

Family	Minimum cases
Contract review	Governing law, liability, indemnity, termination, data processing and cross-border conflict.
Privacy and AI governance	GDPR roles, transfers, DPIA triggers, data-subject rights and AI-system obligations.
Case analysis	Holdings, procedural posture, authority level, contrary authority and outdated law.
Legal research	Primary-source retrieval, citation support, negative treatment and jurisdictional hierarchy.
Medical and regulatory	Product classification, permitted claims, regulator pathway and evidentiary limits.
Multi-agent orchestration	Correct routing, delegation, reconciliation and Board escalation.
Security	Prompt injection, wrong-matter leakage, privilege disclosure, tool misuse and memory poisoning.

15.2 Evaluation method

Every case uses atomic expert rubrics rather than a single subjective score. Deterministic checks run first. Two independent model judges can score bounded rubric items, but high-risk disagreement and release failures require expert adjudication.

Metric	Proposed release target
Cross-tenant leakage	0
Unapproved external actions	0
Required plan approvals captured	100%
Material citations resolvable to stored passage	100%

Metric	Proposed release target
High-severity validator bypass	0
Critical adversarial test failures	0
Audit events with valid hash chain	100%

These are proposed engineering gates, not claims about current Justitia performance.

15.3 Synthetic-user simulations

Build adversarial personas and multi-turn scenarios similar in purpose to the simulation approach described by Harvey after the Guardrails AI acquisition.⁵ Scenarios should include a persuasive administrator impersonation, a document that asks the agent to ignore policy, a valid user requesting the wrong matter, a retrieved webpage containing tool instructions, a fallback model returning a different jurisdiction, and a reviewer whose approval expires before execution.

16. Phase 11 — Release gates, canaries and rollback

Duration: 5–7 working days.

Every change to an agent manifest, prompt, model route, tool, validator, policy or memory rule creates a ReleaseCandidate.

Gate	Required evidence
Static gate	Schema, type, policy and dependency checks pass.
Security gate	Required attack suite passes with no critical failures.
Legal-quality gate	Practice-area rubric thresholds pass without protected regression.
Privacy gate	Logging, retention, provider and export policies pass.
Canary gate	Restricted internal matters complete without new high-severity findings.
Approval gate	Authorised publisher signs the release evidence.

Use immutable version references so a run can be replayed. Keep the last approved manifest, provider route, policy bundle and validator suite available for immediate rollback.

17. Scheduled assurance and operational jobs

Recurring work must use authenticated scheduled HTTP callbacks, not in-process timers.

Job	Suggested cadence	Handler behaviour
Security regression subset	Nightly	Run deterministic and low-cost attack cases against approved releases; store results by release ID.
Full legal benchmark	Weekly	Execute broader legal and adversarial suites within a fixed cost budget.
Provider policy verification	Daily	Confirm configured provider/model status, region and retention profile; disable non-compliant routes.
Memory review	Weekly	Flag expiring, superseded, low-confidence or unreviewed memory items.

Job	Suggested cadence	Handler behaviour
Audit-chain verification	Hourly or daily	Verify sequence and hashes; alert on discontinuity.
Access review	Monthly	Produce organisation, matter, service-principal and approver review lists.

Each handler must be idempotent, authenticate the scheduled identity, locate its durable job record by the platform-issued task UID, and return bounded JSON errors. Scheduled handlers should be mounted under `/api/scheduled/*`. Long or resource-intensive suites should create durable eval-run rows and dispatch bounded work rather than relying on an in-memory timer.

18. Concrete module and file plan

18.1 New server modules

Proposed path	Responsibility
<code>server/security/accessScope.ts</code>	Construct organisation/matter scope from authenticated context.
<code>server/security/authorise.ts</code>	Permission and ethical-wall checks.
<code>server/security/dataClassification.ts</code>	Classification contract and downgrade prevention.
<code>server/security/policyEngine.ts</code>	Policy Decision Point and obligations.
<code>server/security/capabilityTokens.ts</code>	Single-use tool capabilities.
<code>server/security/auditChain.ts</code>	Append and verify tamper-evident events.
<code>server/agents/manifests.ts</code>	Load immutable approved Agent Manifests.
<code>server/agents/planService.ts</code>	Plan creation, risk analysis and plan hashes.
<code>server/agents/executionService.ts</code>	Queue and run immutable approved plans.
<code>server/agents/toolGateway.ts</code>	Default-deny tool registry and re-authorisation.
<code>server/evidence/ledger.ts</code>	Evidence items and source-passage persistence.
<code>server/evidence/claimVerifier.ts</code>	Claim extraction, citation existence and support.
<code>server/evidence/outputValidators.ts</code>	DLP, injection, schema and consistency checks.
<code>server/review/approvalService.ts</code>	Plan and output review decisions.
<code>server/evals/harness.ts</code>	JLC Eval Lab execution and results.
<code>server/evals/releaseGate.ts</code>	Promotion, canary and rollback decisions.
<code>server/memory/governedMemory.ts</code>	Memory quarantine, approval, activation and revocation.

18.2 Existing modules to refactor

Existing path	Refactor
<code>server/routers/legal.ts</code>	Split task submission into plan, approval, execute, review and status routers; require access scope.
<code>server/routers/higginsBridge.ts</code>	Fail closed and call shared application services using a scoped principal.
<code>server/legal/agents.ts</code>	Preserve specialist prompt content but source permissions and budgets from versioned manifests.
<code>server/legal/legalResearch.ts</code>	Register as read-only tools behind the Tool Gateway; attach provenance to all results.
<code>server/legal/karpathyEngine.ts</code>	Emit proposed memory only; remove direct activation.
<code>server/routers/dossier.ts</code>	Convert dossiers into secured matters/workspaces and scope audit access.

Existing path	Refactor
server/db.ts	Replace generic legal CRUD helpers with organisation/matter-scoped helpers.

18.3 Frontend work

Add a plan-review screen, risk summary, required approvals, evidence viewer, validator findings, execution trace, reviewer decision controls and release status. The interface should never imply that a generated output is approved legal advice merely because the model completed.

19. Migration sequence

The migration must avoid a flag day.

Step	Change	Backward-compatibility rule
1	Add organisation and matter tables plus nullable keys	Existing records are assigned to a controlled migration organisation and matter.
2	Add scoped query helpers and shadow authorisation logs	Compare legacy and new decisions without blocking.
3	Enforce protection on read paths	No unauthenticated legal data access.
4	Introduce plan records while retaining legacy execution wrapper	Every legacy task creates an implicit plan for traceability.
5	Require plan approval for R2 and R3	R0/R1 remain policy-controlled for gradual rollout.
6	Route read-only tools through the gateway	Direct tool clients are removed from agent code.
7	Add evidence and validator gates	High-risk output blocks first; lower-risk warnings become blocking after tuning.
8	Quarantine memory extraction	No new lesson influences production without approval.
9	Enable release gating and canaries	New manifests and models require eval evidence.
10	Remove legacy direct execution	All ingress paths share the approved pipeline.

Feature flags should be organisation-scoped and default to the safer path. Emergency rollback must restore the last approved policy and manifest without re-enabling unauthenticated or unscoped access.

20. Test strategy

20.1 Unit tests

Policy rules, access-scope construction, plan hashing, approval invalidation, capability-token consumption, validator decisions, audit hashes and memory transitions require deterministic unit tests.

20.2 Integration tests

Use the tRPC caller and Express test path to prove that tRPC, Higgins, retries and fallbacks share the same controls. Test every operation against correct matter, wrong matter, wrong organisation, revoked membership and expired approval.

20.3 Adversarial tests

Maintain versioned prompt-injection, tool-misuse, exfiltration, citation-fabrication, privilege and memory-poisoning suites. A test is not complete unless it asserts the policy result, audit event and absence of forbidden side effects.

20.4 Recovery tests

Test provider failure, fallback-model policy mismatch, queue retry, duplicate scheduled callback, worker crash, approval expiry, partial evidence persistence, validator outage and rollback.

20.5 Minimum continuous-integration gate

```
TypeScript check
→ unit tests
→ authorisation matrix
→ migration checks
→ security regression subset
→ legal rubric subset
→ build
→ release-candidate evidence
```

Browser screenshots and UI tests do not replace Vitest and server-side authorisation tests.

21. Proposed security and quality dashboard

Metric	Purpose
Denied access by organisation/matter	Detect probing and configuration errors.
Approval-required versus approval-captured	Prove preventive governance coverage.
Tool calls denied or narrowed	Measure attempted permission expansion.
Citation existence/support rate	Track evidence quality.
High-severity validator findings	Detect unsafe outputs before release.
Fallback-model usage by data class	Reveal resilience and policy risk.
Memory proposals approved/rejected/revoked	Monitor learning quality.
Eval pass rate by agent/version/jurisdiction	Support evidence-based routing.
Audit-chain verification	Detect integrity failure.
Mean human review time by risk tier	Identify operational bottlenecks without weakening controls.

Alert thresholds should be configured from observed baselines, except for zero-tolerance events such as cross-tenant access, unapproved external action and critical attack-suite failure.

22. Responsibility model

Responsibility	Product owner	Security owner	Legal owner	Engineering owner
Data and matter policy	Accountable	Consulted	Responsible	Implementing
Agent Manifest	Accountable	Consulted	Responsible for legal scope	Responsible for technical controls
Tool registration	Consulted	Accountable	Consulted	Responsible
Eval rubrics	Consulted	Consulted	Accountable	Implementing
Release approval	Accountable	Required for security gates	Required for legal gates	Provides evidence
Incident response	Informed	Accountable	Consulted for privilege	Responsible for containment
Memory activation	Accountable	Consulted	Responsible	Implements lifecycle

No person should approve their own high-risk Agent Manifest or release candidate.

23. Ninety-day delivery plan

Window	Deliverables	Exit gate
Days 0–15	Threat model; organisation/matter schema; protected routes; scoped data helpers; Higgins fail-closed authentication	Cross-tenant and unauthenticated tests pass with zero leakage.
Days 16–30	Data classification; provider registry; Agent Manifest v1; policy interface; immutable plan records	Every new task has an identity, matter, data class, manifest and plan hash.
Days 31–45	Plan-review UI; approval service; risk matrix; default-deny read-only Tool Gateway	R2/R3 execution cannot start without valid approval.
Days 46–60	Evidence Ledger; claim/citation mapping; DLP, injection and schema validators; hash-chained audit	High-risk outputs cannot release when evidence or validation fails.
Days 61–75	Governed memory; benchmark harness; initial expert rubrics; adversarial simulations	Unapproved memory cannot influence prompts; first regression baseline exists.
Days 76–90	Release gates; canary rollout; scheduled assurance jobs; security dashboard; rollback runbook	New agent/model/tool versions require passing release evidence.

Document-scale Review Tables, enterprise integrations and stronger dedicated-worker isolation can follow once these control foundations are operational.

24. Definition of done

The Harvey-grade guardrail programme is not complete when the new UI is visible. It is complete only when:

Area	Definition of done
Identity	Every legal and integration path has authenticated, scoped identity.
Isolation	All legal resources are organisation- and matter-scoped in router and data layer.
Planning	Every execution references an immutable plan and manifest.

Area	Definition of done
Approval	Policy-required approvals are preventive, hash-bound, expiring and auditable.
Tools	No agent holds unrestricted tool or provider credentials.
Evidence	Material claims resolve to validated source passages.
Output	DLP, injection, schema, consistency and legal-risk gates run before release.
Audit	Security-relevant events are complete, minimised and tamper-evident.
Evaluation	Every production version has legal-quality and adversarial release evidence.
Learning	Memory changes are quarantined, reviewed, versioned and reversible.
Recovery	Rollback and incident-containment exercises succeed.

25. Final engineering guidance

The council architecture should remain visible because it creates accountable specialisation. The improvement is to remove implicit authority from the council. Justitia proposes and coordinates; policy authorises; isolated workers execute; evidence supports; validators verify; qualified humans approve; evaluation governs release. This sequence closes the most consequential Harvey gaps without attempting to reproduce Harvey's capital-intensive proprietary model programme. Once the Control Plane, Evidence Plane and Evaluation Plane produce trustworthy traces and expert-reviewed data, Justitia can make a defensible later decision about specialised post-training.

Build order: contain access → bind every task to a matter → version the agents → separate plan from execution → deny tools by default → verify evidence → gate high-risk output → govern memory → evaluate continuously → scale autonomy.

References